

Scientific Computation

Federico Formica

Material by: Prof. Matthew Giamou

Nov. 10, 2025

Lecture 24 - Polynomial Interpolation

Clarification: any basis function?

In general, we can perform interpolation with any set of basis functions.

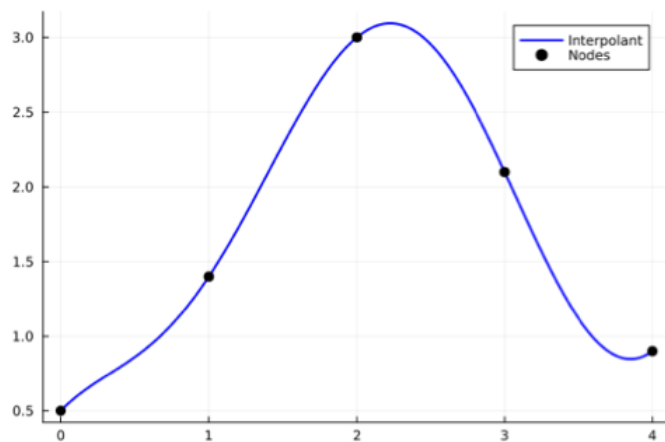
$$\nu(x) = \sum_{i=0}^n c_i \phi_i(x) = c_0 \phi_0(x) + c_1 \phi_1(x) + \dots + c_n \phi_n(x)$$
$$Xc = \begin{bmatrix} \phi_0(x_0) & \phi_1(x_0) & \dots & \phi_n(x_0) \\ \phi_0(x_1) & \phi_1(x_1) & \dots & \phi_n(x_1) \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \end{bmatrix}$$

$$\begin{bmatrix} \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_n) & \phi_1(x_n) & \cdots & \phi_n(x_n) \end{bmatrix} \begin{bmatrix} \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} \vdots \\ y_n \end{bmatrix}$$

Does this work for any set of (linearly independent) basis functions?

2

Clarification: any basis function?



```

1 # Define nodes
2 x = [0.0; 1.0; 2.0; 3.0; 4.0];
3 y = [0.5; 1.4; 3.0; 2.1; 0.9];
4
5 # Define Vandermonde matrix
6 X = [ones(length(x),1) x x.^2 sin(2.* x) exp(0.1.* x)];
7
8 # Find interpolant coefficients
9 c = X \ y;
10
11 # Define interpolant
12 v(x) = @. c[1]+c[2]*x+c[3]*x^2+c[4]*sin(2*x)+c[5]*exp(0.1*x);
13 c_round = round.(c,sigdigits = 4)
14 println("Interpolant function:")
15 println("v(x) = $(c_round[1])$(c_round[2])*x$(c_round[3])*x^2")
16
17 # Plot interpolant and nodes
18 using Plots
19 x_plot = minimum(x):0.01:maximum(x)
20 plot(x_plot,v(x_plot),color=:blue,lw=2,label="Interpolant")
21 scatter!(x,y,markersize=5,color=:black,label="Nodes")

```

Interpolant function:
 $v(x) = -792.1 - 76.5x - 5.158x^2 - 0.8735\sin(2x) + 792.6\exp(0.1x)$

3

Lecture Outline

- Vandermonde matrix
- Lagrange interpolation
- Newton's form

Vandermonde Matrix

In general, we have the linear system

$$V(x_i) = y_i$$

$$Xc = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix},$$

where the coefficient matrix X is called a **Vandermonde matrix**.

Uniqueness of the Interpolant

Therefore, if the x_i are all unique, then the determinant

$$\det(X) = \prod_{i=0}^n \left[\prod_{j=i+1}^n (x_j - x_i) \right] \neq 0,$$

and we know that:

Lecture Outline

- Vandermonde matrix

- Lagrange interpolation
- Newton's form

Lagrange Basis Functions

Let's explore an alternative polynomial basis. Define the **Lagrange basis functions** L_j for $j = 0, 1, \dots, n$ as satisfying

$$\phi_i \checkmark \quad L_j(x_i) = \begin{cases} 0, & \text{if } i \neq j \\ 1, & \text{if } i = j. \end{cases}$$

and our interpolant is now the **Lagrange polynomial**:

$$p(x) = \sum_{j=0}^n y_j L_j(x).$$

At data point x_i :

$$\begin{aligned} p_n(x_i) &= \sum_{j=0}^n y_j L_j(x_i) = \sum_{j=0}^{i-1} y_j \cancel{L_j(x_i)} + \underbrace{y_i L_i(x_i)}_1 + \sum_{j=i+1}^n y_j \cancel{L_j(x_i)} \\ &= y_i \end{aligned}$$

Lagrange Basis Functions

What polynomials $L_j(x)$ satisfy the requirement

$$L_j(x_i) = \begin{cases} 0, & \text{if } i \neq j \\ 1, & \text{if } i = j. \end{cases}$$

numerator
normalizer

$$L_j(x) = \frac{(x-x_0)(x-x_1)\dots(x-x_{j-1})(x-x_{j+1})\dots(x-x_n)}{(x_j-x_0)(x_j-x_1)\dots(x_j-x_{j-1})(x_j-x_{j+1})\dots(x_j-x_n)}$$

$$= \prod_{\substack{i=0 \\ i \neq j}} \frac{(x-x_i)}{(x_j-x_i)}$$

9

A Small Lagrange Polynomial

Example 2: Write the Lagrange polynomial of the dataset

	x_0	x_1	x_2
x_i :	1	2	4
y_i :	1	3	3

$\phi_0(x) = 1$
 $\phi_1(x) = x$
 $\phi_2(x) = x^2$

for Linear regression

$$L_0(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} = \frac{(x-2)(x-4)}{(1-2)(1-4)} = \frac{x^2 - 6x + 8}{3}$$

$$L_1(x) = \frac{(x-1)(x-4)}{(2-1)(2-4)} = -\left(\frac{x^2 - 5x + 4}{2}\right)$$

$$L_2(x) = \frac{(x-1)(x-2)}{(4-1)(4-2)} = \frac{x^2 - 3x + 2}{6}$$

check $L_i(x_i) = 1$, $L_i(x_j) = 0$ for $i \neq j$

10

A Nice Linear System

Example 2 cont.: What does the corresponding linear system look like?

$$\begin{bmatrix} \phi_0(x_0) & \phi_1(x_0) & \cdots & \phi_n(x_0) \\ \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_n(x_1) \\ \vdots & \vdots & & \vdots \\ \phi_0(x_n) & \phi_1(x_n) & \cdots & \phi_n(x_n) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \underbrace{\begin{bmatrix} L_0(x_0) & L_1(x_0) & \cdots & L_n(x_0) \\ L_0(x_1) & L_1(x_1) & \cdots & L_n(x_1) \\ \vdots & \vdots & & \vdots \\ L_0(x_n) & L_1(x_n) & \cdots & L_n(x_n) \end{bmatrix}}_{L=I} c = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

We know $c=y$, so $L=I$!

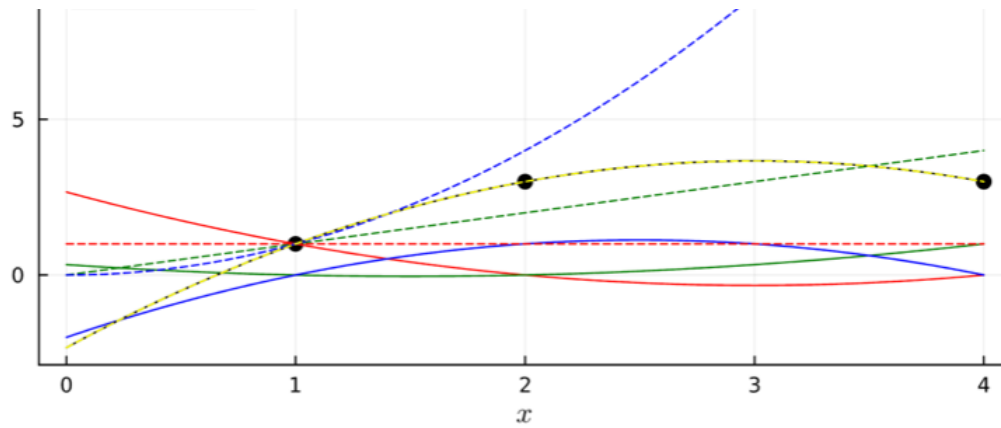
$L_i(x_i) = 1$ (diagonal), $L_i(x_j) = 0$ for $i \neq j$ (off diagonal)

so "solving" for $c=y$ is trivial for the Lagrangian basis, the work is now in defining/evaluating the basis polynomial $L_i(x)$

11

Example 2 Visualized





We have therefore just used a different **basis** that spans the same space of polynomials!
This is the **Lagrange form** of the **same** interpolation polynomial!

12

Lecture Outline

- Vandermonde matrix
- Lagrange interpolation
- **Newton's form**

The Newton Form

Let's examine one last choice of basis, leading to yet another way of solving for the interpolating polynomial. Let our basis functions be

$$\phi_j(x) = \prod_{i=0}^{j-1} (x - x_i) = (x - x_0)(x - x_1) \cdots (x - x_{j-1}), \quad j = 0, \dots, n.$$

$\rightarrow n=4$

For a cubic interpolant, this looks like:

$$\phi_0(x) = 1$$

$$\phi_1(x) = (x - x_0)$$

$$\phi_2(x) = (x - x_0)(x - x_1) = (x - x_1)\phi_1(x)$$

$$\phi_3(x) = (x - x_2)\phi_2(x)$$

14

Computing Coefficients

Let $\nu(x)$ denote our interpolant, which is a degree n polynomial:

$$\nu(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots + c_n(x - x_0)(x - x_1) \cdots (x - x_{n-1})$$

At data points x_i we need to enforce

$$\nu(x_i) = c_0 + c_1(x_i - x_0) + c_2(x_i - x_0)(x_i - x_1) + \dots + c_n(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{n-1}) = y_i$$

At $x = x_0$ we have:

$$p_n(x_0) = c_0 + c_1(\cancel{x_0 - x_0}) + c_2(\cancel{x_0 - x_0})(x_0 - x_1) \cdots = y_0$$

$$\Rightarrow c_0 = y_0$$

15

Computing c_1

At $x = x_1$:

$$\nu(x_1) = c_0 + c_1(x_1 - x_0) + c_2(x_1 - x_0)(x_1 - x_1) + \dots c_n(x_1 - x_0)(x_1 - x_1) \cdots (x_1 - x_{n-1}) = y_1$$

$$= c_0 + c_1(x_1 - x_0) = y_1$$

$$\Rightarrow c_1 = \frac{y_1 - c_0}{x_1 - x_0} \Leftrightarrow \frac{y_1 - y_0}{x_1 - x_0}$$

16

Computing c_2

At $x = x_2$:

$$\nu(x_2) = c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1) + \dots c_n(x_2 - x_0)(x_2 - x_1) \cdots (x_2 - x_{n-1}) = y_2$$

$$= y_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1) = y_2$$

$$c_2 = \frac{y_2 - y_0 - \left(\frac{y_1 - y_0}{x_1 - x_0}\right)(x_2 - x_0)}{(x_2 - x_0)(x_2 - x_1)}$$

17

Computing c_2 Continued

$$c_2 = \frac{y_2 - y_0 - \frac{y_1 - y_0}{x_1 - x_0}(x_2 - x_0)}{(x_2 - x_0)(x_2 - x_1)}$$

do this to get the form for c_n that we want

$$= \frac{y_2 [-y_1 + y_1] - y_0 - \frac{y_1 - y_0}{x_1 - x_0}(x_2 - x_0)}{(x_2 - x_0)(x_2 - x_1)}$$

$$= \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{(y_1 - y_0)(x_2 - x_0)}{(x_1 - x_0)(x_2 - x_1)} + \frac{y_1 - y_0}{x_2 - x_1}}{x_2 - x_0}$$

18

Computing c_2 Continued

$$c_2 = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{(y_1 - y_0)(x_2 - x_0)}{(x_1 - x_0)(x_2 - x_1)} + \frac{y_1 - y_0}{x_2 - x_1}}{(x_2 - x_0)}$$

$$= \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0} \left(\frac{x_2 - x_0}{x_2 - x_1} - \frac{x_1 - x_0}{x_2 - x_1} \right)}{x_2 - x_0} \quad \rightarrow \quad |$$

$$= \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} = c_2$$

19

Divided Differences

$$y_0, y_1, \dots, y_n$$

Given $x_0, x_1, \dots, x_n$ where $0 \leq i < j \leq n$, recursively define

$$[y_i, \dots, y_j] = \frac{[y_{i+1}, \dots, y_j] - [y_i, \dots, y_{j-1}]}{x_j - x_i},$$

$$[y_i] \triangleq y_i$$

where $[y_i, \dots, y_j]$ are called **divided differences** over $x_i, \dots, x_j$.

↳
This notation lets us simplify the expressions for c_n in the

Newton polynomial

20

Coefficients from Divided Differences

Using this definition, we can write

$$c_0 = y_0 = [y_0] = y_0$$

$$c_1 = \frac{y_1 - y_0}{x_1 - x_0} = [y_0, y_1] = \frac{[y_1] - [y_0]}{x_1 - x_0}$$

$$c_2 = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} = \frac{[y_1, y_2] - [y_0, y_1]}{x_2 - x_0} = [y_0, y_1, y_2]$$

$\vdots$

$$c_n = \frac{[y_1, \dots, y_n] - [y_0, \dots, y_{n-1}]}{x_n - x_0} = [y_0, y_1, \dots, y_n]$$

21

Computing Newton Coefficients

Example 1: Determine the Newton polynomial for the dataset

$$\begin{array}{lcl} x_i : & 1 & 2 & 4 \\ y_i : & 1 & 3 & 3 \end{array}$$

i	x_i	$[y_i] = y_i$	$[y_{i-1}, y_i]$	$[y_{i-2}, y_{i-1}, y_i]$
0	1	1	N/A	N/A
1	2	3	$[y_0, y_1]$	N/A
2	4	3	$[y_1, y_2]$	$[y_0, y_1, y_2]$

Computing Newton Coefficients

Example 1 cont.: Let's compute the coefficients by hand:

i	x_i	$[y_i] = y_i$	$[y_{i-1}, y_i]$	$[y_{i-2}, y_{i-1}, y_i]$
0	1	1	N/A	N/A
1	2	3	$[y_0, y_1]$	N/A
2	4	3	$[y_1, y_2]$	$[y_0, y_1, y_2]$

Computing Newton Coefficients

Example 1 cont.: Suppose we add in a **fourth point**:

$$\begin{array}{l} x_i : \quad 1 \quad 2 \quad 4 \quad \color{red}{3} \\ y_i : \quad 1 \quad 3 \quad 3 \quad \color{red}{5} \end{array}$$

Can we leverage our pre-computed results for the first three points? Let's look at the definition of our Newton polynomial basis:

$$\nu(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots c_n(x - x_0)(x - x_1)\dots(x - x_{n-1})$$

And a reminder that our coefficients are:

$$c_n = \frac{[y_1, \dots, y_n] - [y_0, \dots, y_{n-1}]}{x_n - x_0} = [y_0, y_1, \dots, y_n]$$

24

Computing Newton Coefficients

Example 1 cont.: The old coefficients remain the same!

$$\begin{array}{l} x_i : \quad 1 \quad 2 \quad 4 \quad \color{red}{3} \\ y_i : \quad 1 \quad 3 \quad 3 \quad \color{red}{5} \end{array}$$

i	x_i	$[y_i] = y_i$	$[y_{i-1}, y_i]$	$[y_{i-2}, y_{i-1}, y_i]$	$[y_{i-3}, y_{i-2}, y_{i-1}, y_i]$
0	1	1	N/A	N/A	N/A
1	2	3	2	N/A	N/A
2	4	3	0	$-\frac{2}{3}$	N/A

3

3

5

上一话我们讲到了monomial polynomial做插值太麻烦了，我们不妨换个思路，从basis function入手来简化系数c。

我们定义了拉格朗日基函数，

$$L_j(x_i) = \begin{cases} 0, & \text{if } i \neq j \\ 1, & \text{if } i = j \end{cases}$$

所以这就把interpolation函数的系数变成了 y_i ，就不用求了！

它是如何被“构造”出来的？

这个公式的构造分为两步，完美地对应了“0”和“1”这两个要求：

1. 如何满足 $L_j(x_i) = 0$? (手写的 "numerator")

- **目标：** 我们需要 $L_j(x)$ 在**所有**其他数据点 x_i ($i \neq j$) 处都等于 0。
- **方法：** 在代数中，要让一个多项式在 x_0 处等于 0，你只需要让它包含 $(x - x_0)$ 这一项。
- **构造：** 为了让 $L_j(x)$ 在 $x_0, x_1, \dots, x_{j-1}, x_{j+1}, \dots, x_n$ 处都等于 0，我们只需要把这些项**全部乘在一起**。
- **结果：** 这就构成了幻灯片中的**分子 (numerator)**： $(x - x_0)(x - x_1) \dots (x - x_{j-1})(x - x_{j+1}) \dots (x - x_n)$
- (现在我们有了一个“几乎”正确的函数，它在所有 $i \neq j$ 的 x_i 处都为 0。)

2. 如何满足 $L_j(x_j) = 1$? (手写的 "normalizer")

- **问题：** 我们用上面那个分子构造的函数，在 x_j 处的值是多少？
 - 把它代入： $(x_j - x_0)(x_j - x_1) \dots (x_j - x_{j-1}) \dots$
 - 这个值**不是 1**，而是一个我们不想要的、乱七八糟的常数。
- **方法：** 我们用一个技巧，把这个函数**除以**它在 x_j 处的“错误”值。这个操作就叫***“归一化”

(Normalization)**。

- **构造：** 我们把上面那个“乱七八糟的常数”放到分母里。
- **结果：** 这就构成了幻灯片 中的**分母 (denominator)**，手写笔记称之为***“归一化项” (normalizer)**： $(x_j - x_0)(x_j - x_1) \dots (x_j - x_{j-1}) \dots (x_j - x_n)$
- (现在，当我们把 x_j 代入整个公式时，分子和分母会变得一模一样，它们相除的结果**正好就是 1。**)

总结

- **公式 $L_j(x)$ ：** 通过组合这两步，我们得到了完整的拉格朗日基函数公式。
- **红色手写笔记 II：** 这是该公式的“紧凑”写法，使用了**连乘符号 (Product notation)**。
 - $\prod_{i=0, i \neq j}^n$
 - **翻译：** “把 i 从 0 循环到 n ，但**跳过 $i = j$ 那一项**，然后把所有的 $\frac{(x-x_i)}{(x_j-x_i)}$ 这一项**乘起来**。”

这张幻灯片 向您展示了 $L_j(x)$ 并不是凭空冒出来的，而是通过“**先造 0，再归 1**”的逻辑，被巧妙地构造出来的。

所以使用拉格朗日基函数的时候不要求解系数，我们把工作量转移到设置拉格朗日基函数上了。

牛顿插值法是一种新的方法，**他解决了之前插值方法不好添加新点的问题；**

1. 新的基函数 $\phi_j(x)$ ：

- 它定义了一组新的、“**嵌套**”的 (nested) 基函数。
- $\phi_j(x) = \prod_{i=0}^{j-1} (x - x_i)$ ，意思是“从 x_0 开始，把 $(x - x_i)$ 乘起来，直到 x_{j-1} 为止”。

2. 手写的例子 (cubic = $n = 3$)：

- $\phi_0(x) = 1$ ($j = 0$ ，连乘是空的，按惯例为 1)
- $\phi_1(x) = (x - x_0)$ ($j = 1$ ，只乘 $i = 0$ 这一项)
- $\phi_2(x) = (x - x_0)(x - x_1)$ ($j = 2$ ，乘 $i = 0, 1$ 两项)
- $\phi_3(x) = (x - x_0)(x - x_1)(x - x_2)$
- **递归定义：** 正如幻灯片 所写， $\phi_2(x) = \phi_1(x) \cdot (x - x_1)$ ， $\phi_3(x) = \phi_2(x) \cdot (x - x_2)$

为什么这个基函数如此“聪明”？

这个“嵌套”定义的核心***“回报”**在于它对 $Xc = y$ 那个线性系统 X 矩阵的结构**所产生的影响。**

让我们来看看 X 矩阵 (其中 $X_{ij} = \phi_j(x_i)$) 会变成什么样子：

- 第 j 列 (基函数): $\phi_j(x) = (x - x_0) \dots (x - x_{j-1})$
- 第 i 行 (数据点): x_i

现在我们来计算 $X_{ij} = \phi_j(x_i)$:

- $\phi_j(x_i) = (x_i - x_0)(x_i - x_1) \dots (x_i - x_{j-1})$

关键洞察:

- 当 $i < j$ 时 (例如, 计算第 3 行第 4 列的元素 $X_{34} = \phi_4(x_3)$):
 - $\phi_4(x_3) = (x_3 - x_0)(x_3 - x_1)(x_3 - x_2)(x_3 - x_3)$
 - 因为 $(x_3 - x_3) = 0$, 所以整个乘积都为 0!
- **结论:** 只要 $i < j$, X_{ij} 必定为 0。

这意味着 X 矩阵是一个下三角矩阵 (Lower Triangular Matrix)!

$$X = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ \phi_0(x_1) & \phi_1(x_1) & 0 & \dots & 0 \\ \phi_0(x_2) & \phi_1(x_2) & \phi_2(x_2) & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_n) & \phi_1(x_n) & \phi_2(x_n) & \dots & \phi_n(x_n) \end{bmatrix}$$

这种形式的好处 (“回报”)

1. 求解速度快:

- 求解一个稠密矩阵 $Xc = y$ (范德蒙方法) 需要 $O(n^3)$ 的时间。
- 求解一个三角矩阵 $Xc = y$ (牛顿形式) 只需要**“向前代入法” (Forward-Substitution)**，时间复杂度仅为 $O(n^2)$ 。这快得多!

2. 易于更新 (可扩展性):

- 这是最大的优点, 它直接解决了您在 [image_09e3ed.jpg](#) 中看到的“难以添加新点”的问题。
- 如果您已经解出了 $n = 10$ 个点的 $c_0, \dots, c_9$, 现在想添加一个新点 (x_{10}, y_{10}) :
- 您的 10×10 三角矩阵 X 和系数 $c_0, \dots, c_9$ **完全不变!**
- 您只需要添加一行和一列, 然后解一个新方程来找到一个新系数 c_{10} 。
- (这正是您在很早之前 [image_066ea1.jpg](#) 幻灯片上看到的“利用预计算结果”的特性!)